Recursion

- Recursion is when a function calls itself.
- Recursions need a base condition.
- Problems that can be broken into smaller identical problems can often be solved recursively.
- Sometimes recursive solutions end up computing the same results many times…

Defining and calling functions

Run the statements as they are defined

```
def greeting(name):
    salute = "Hello"
    return salute + "  " + name


message = greeting("Mogens")
print(message)
```

Factorial

```
4! : 4 * 3 * 2 * 1
```

```
f(4) : 4 * 3 * 2 * 1
```

f(3)

f(2)

Factorial

`4! : 4 * 3 * 2 * 1`

Recursion: `f(x) = x * f(x-1)`

Basis: `f(1) = 1`

Exercise

Write a Python function that computes the factorial recursively:

Recursion: `f(x) = x * f(x-1)`

Basis: `f(1) = 1`

1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
9	362880
10	3628800
11	39916800
12	479001600
13	6227020800
14	87178291200
15	1307674368000
16	20922789888000
17	355687428096000
18	6402373705728000
19	121645100408832000
20	2432902008176640000

Solution

```
def factorial(x):
    if x == 1:
        return 1
    return x * factorial(x-1)
```

1

```
def factorial(x):
    if x == 1:
        return 1
    return x *
```

2

```
def factorial(x):
    if x == 1:
        return 1
    return x *
```

6

result =

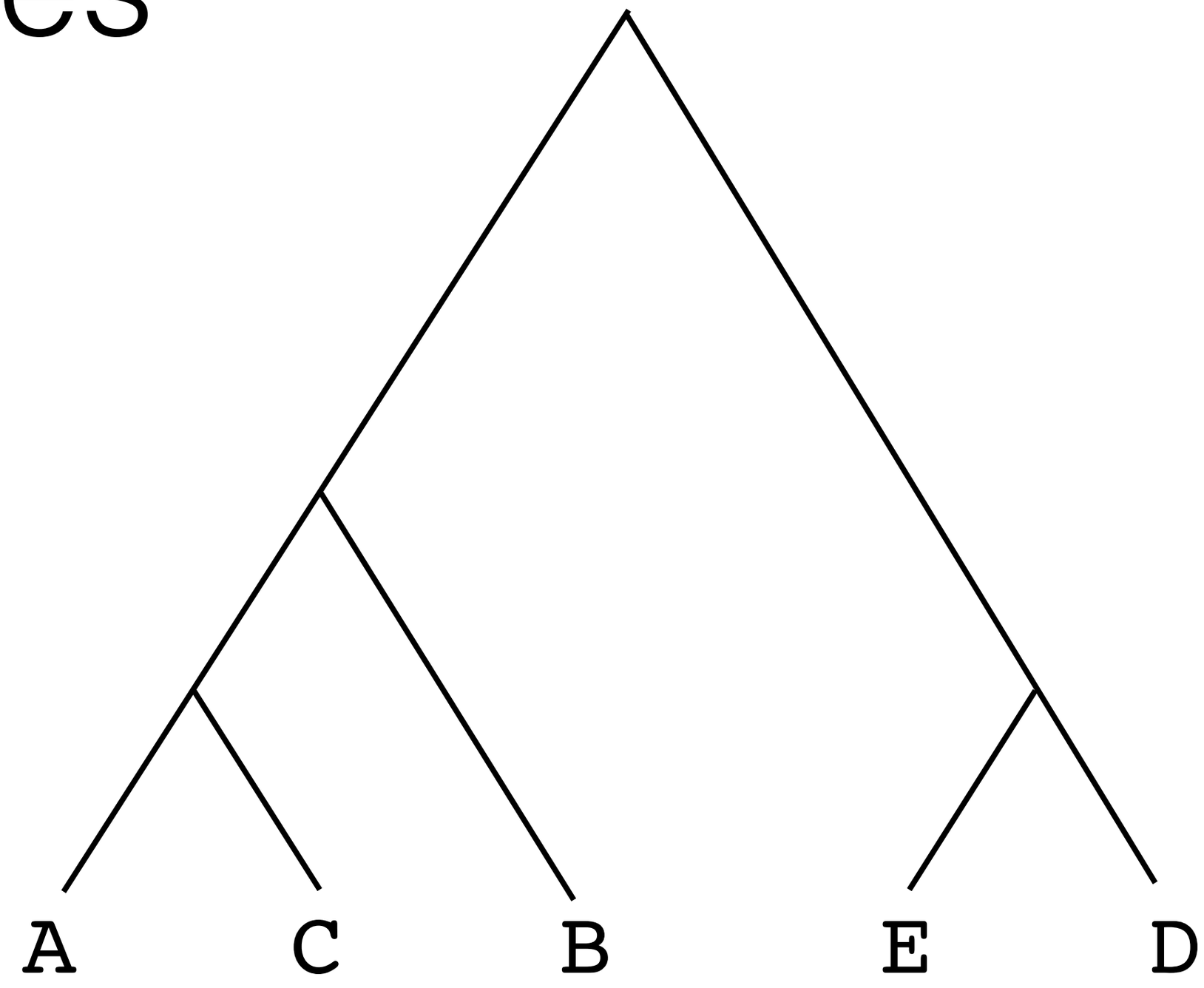
Try:

```
factorial(100)
```

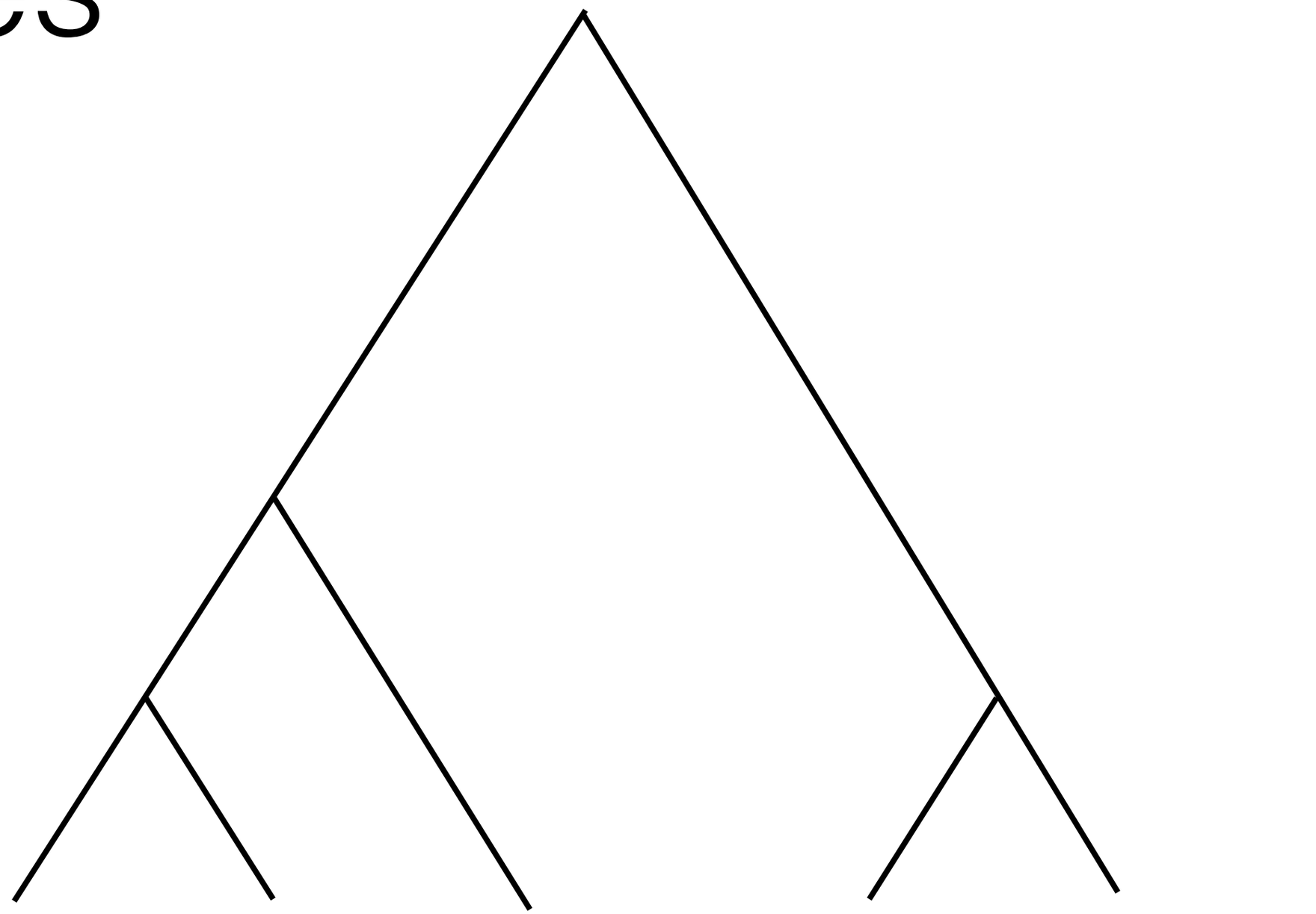
And

```
factorial(1000)
```

Trees

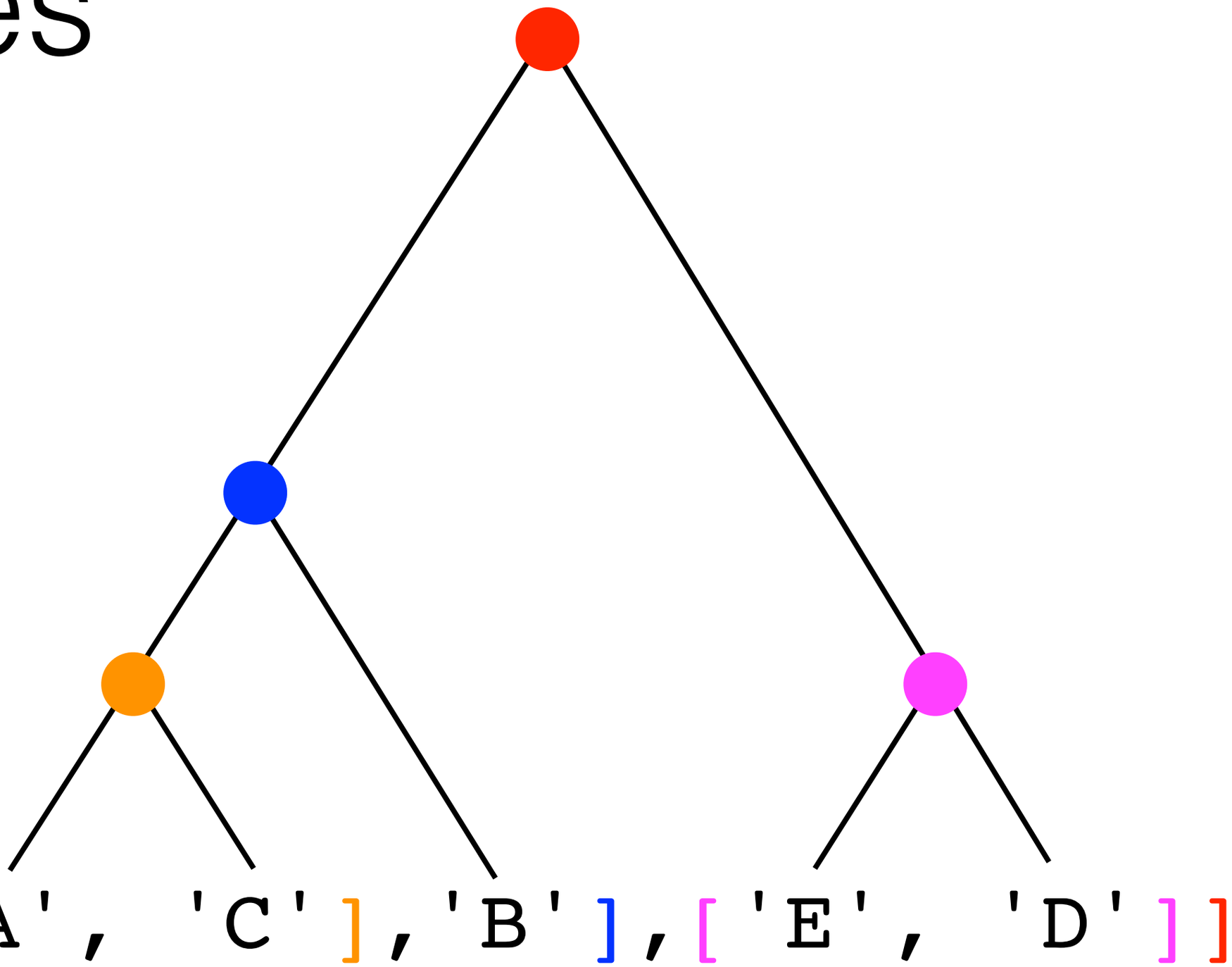


Trees



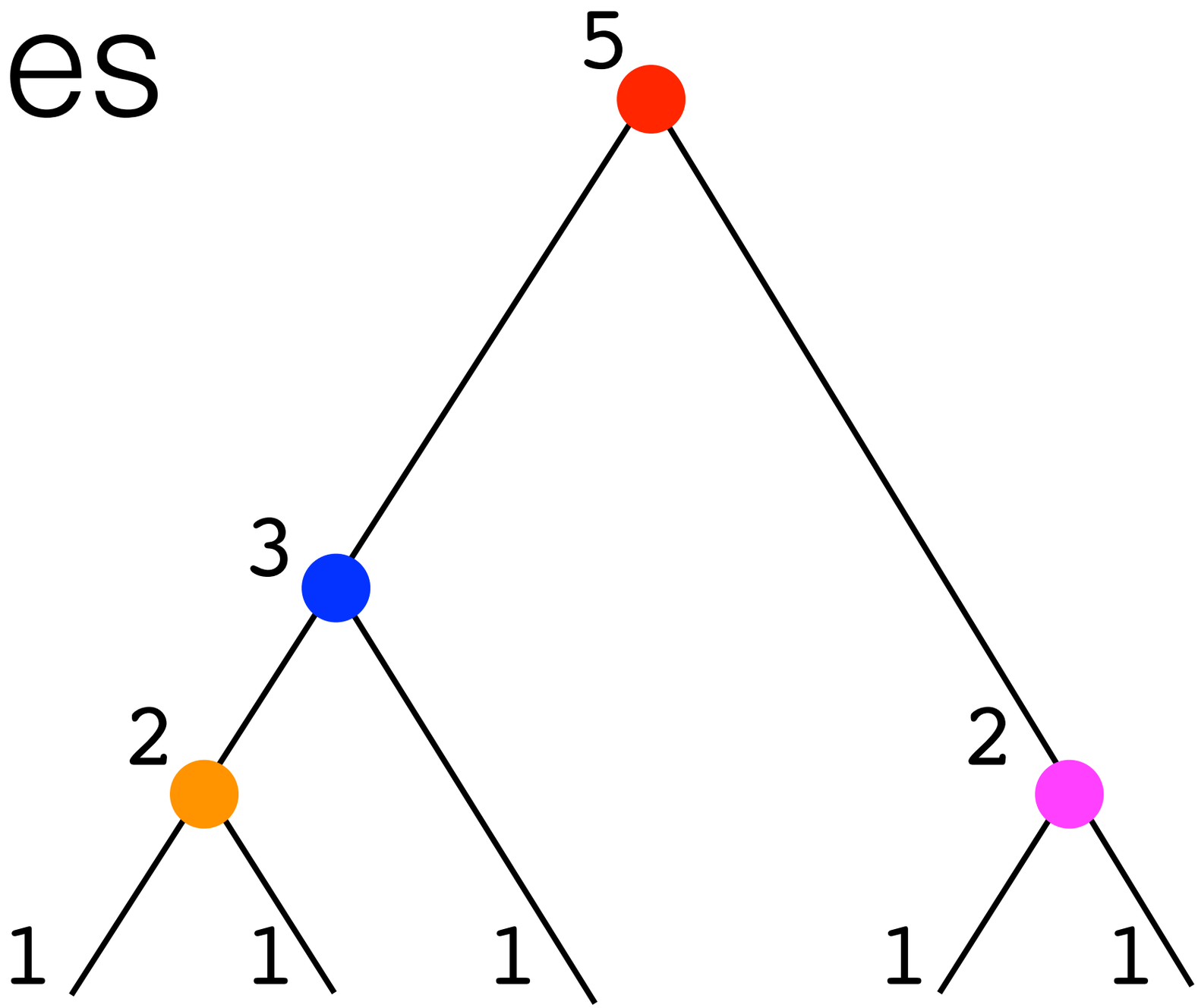
[[['A', 'C'],'B'],['E', 'D']]

Trees



[[['A', 'C'],'B'],['E', 'D']]

Trees



Exercise

- **How can we find the number of leaves on a tree using recursion?**
- How can we divide the problem into smaller identical problems?
- What is the base case that has a trivial solution?

Exercise

```
[[['A', 'C'],'B'],['E', 'D']]
```

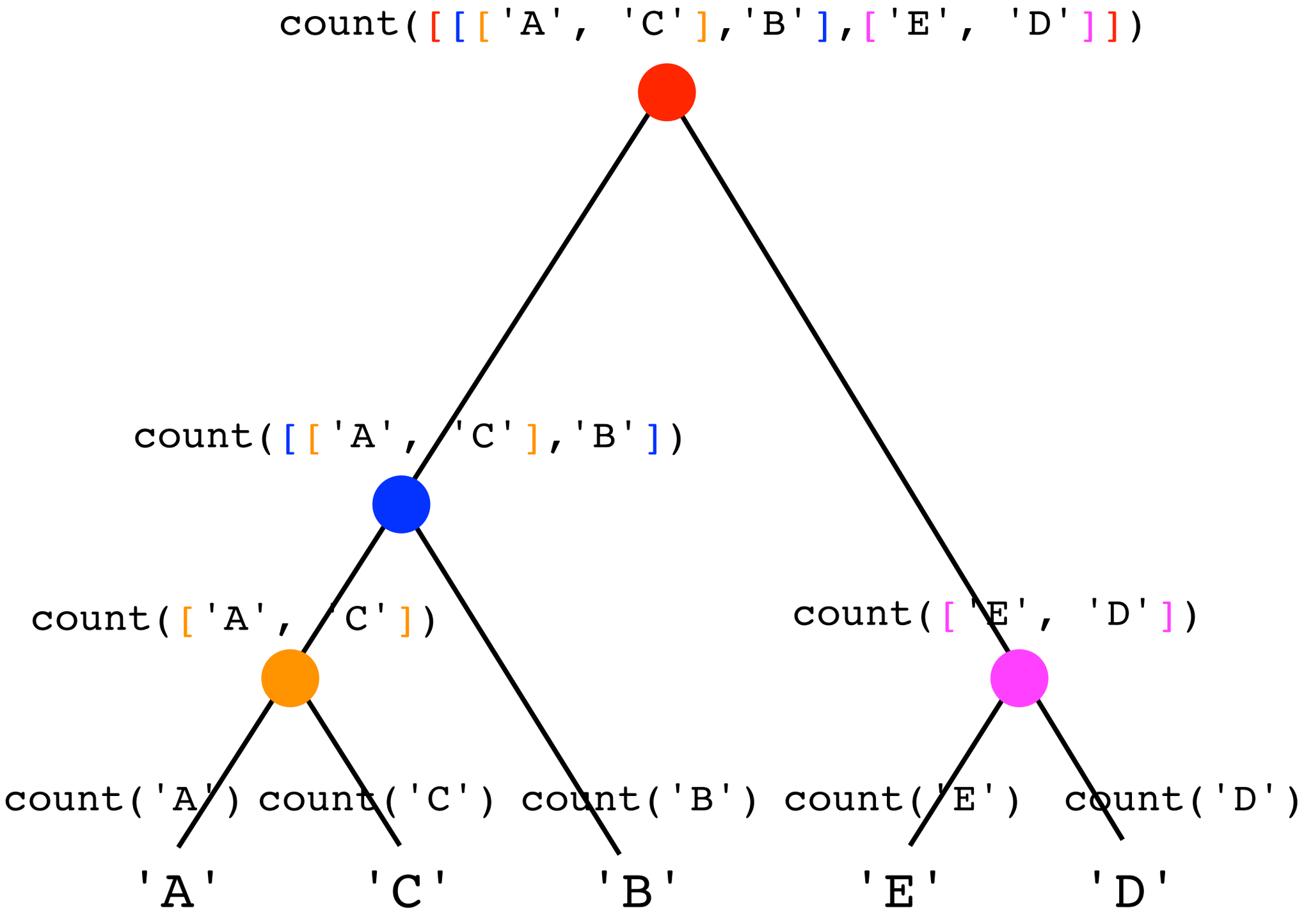
- Write a function that returns the number of leaves on a tree.

- Hint: test if a value is a string like this:

```
if type(tree) is str:
    statements…
```

Løsning

```
def count(tree):
    if type(tree) is str:
        return 1
    return count(tree[0]) + count(tree[1])
```



Exercise

```
[[['A', 'C'],'B'],['E', 'D']]
```

- Write a function that returns the **a list of all leaves** on a tree.

- Hint: test if a value is a string like this:

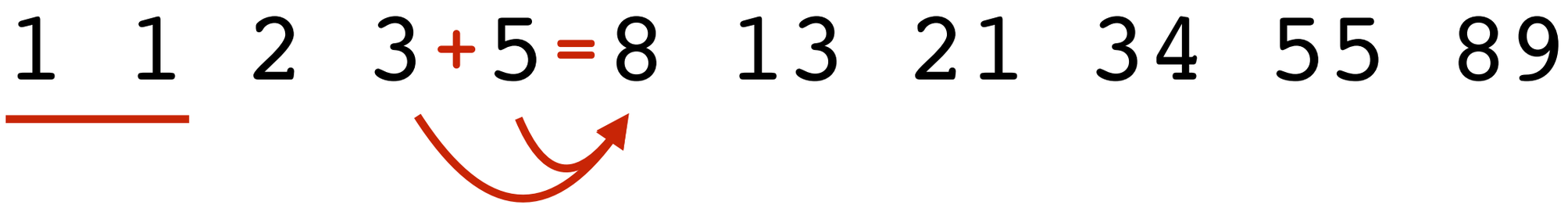
```
if type(tree) is str:
    statements…
```

```
MAT = {'A': {'A': 2, 'T': 0, 'G': 0, 'C': 0},
       'T': {'A': 0, 'T': 2, 'G': 0, 'C': 0},
       'G': {'A': 0, 'T': 0, 'G': 2, 'C': 0},
       'C': {'A': 0, 'T': 0, 'G': 0, 'C': 2}}

G = -1

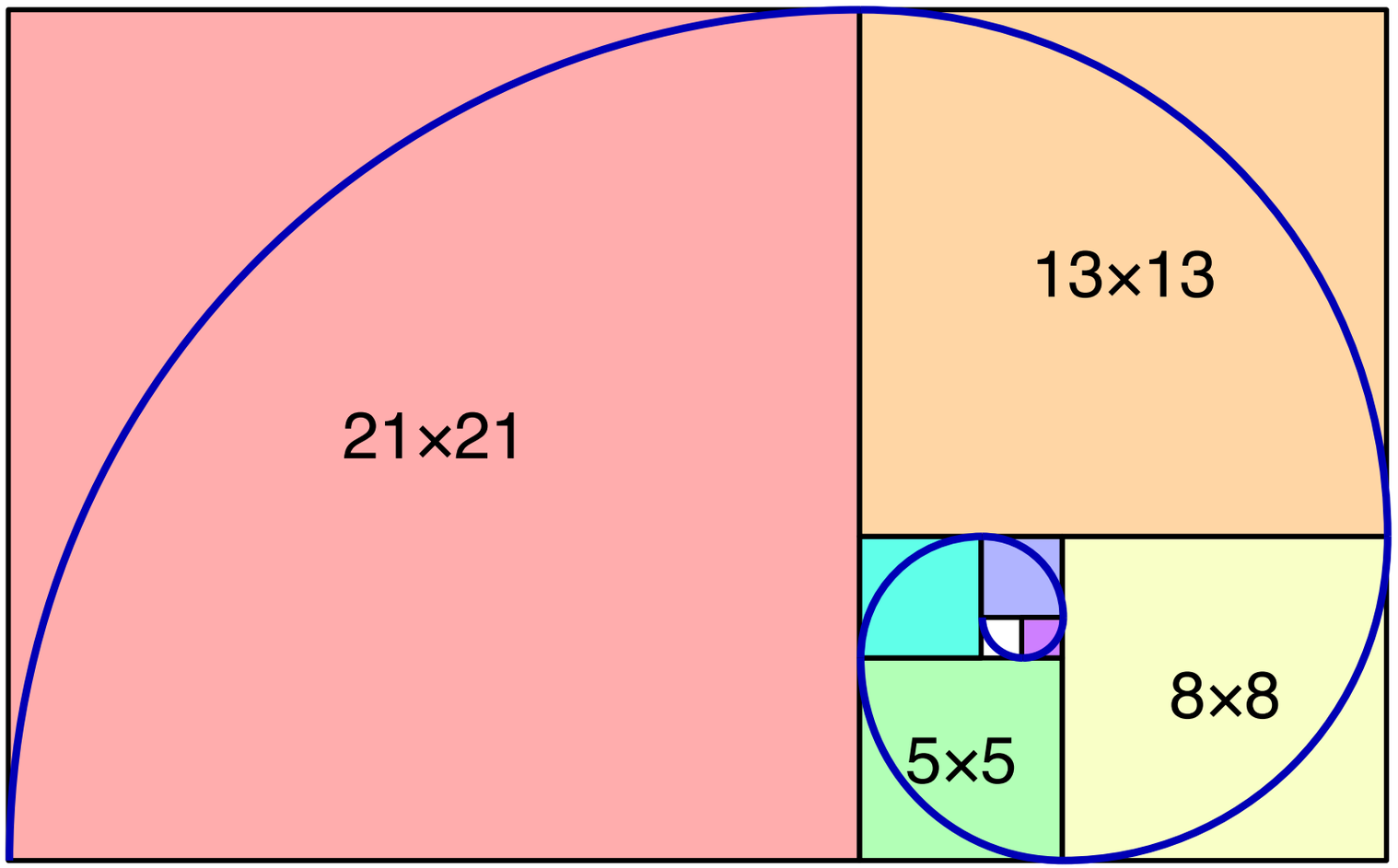
def score(a, b):
    if len(a) == 0 and len(b) == 0:
        return 0
    possible = []
    if len(a) > 0 and len(b) > 0:
        s = score(a[:-1], b[:-1]) + MAT[a[-1]][b[-1]]
        possible.append(s)
    if len(a) > 0:
        s = score(a[:-1], b) + G
        possible.append(s)
    if len(b) > 0:
        s = score(a, b[:-1]) + G
        possible.append(s)
    return max(possible)
```

Fibonacci numbers



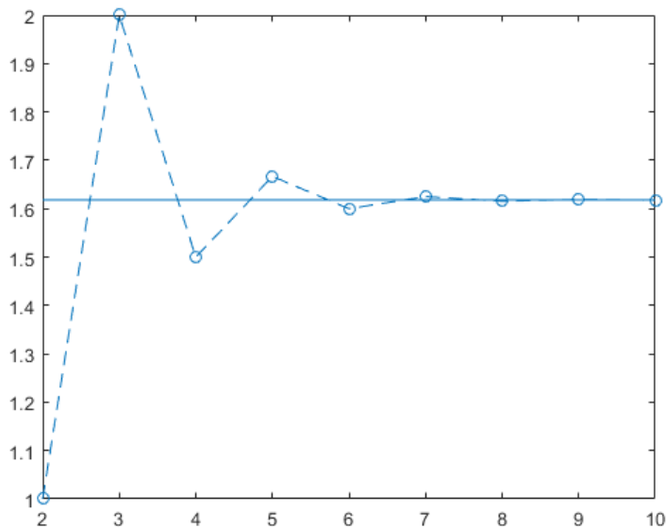
Fibonacci numbers

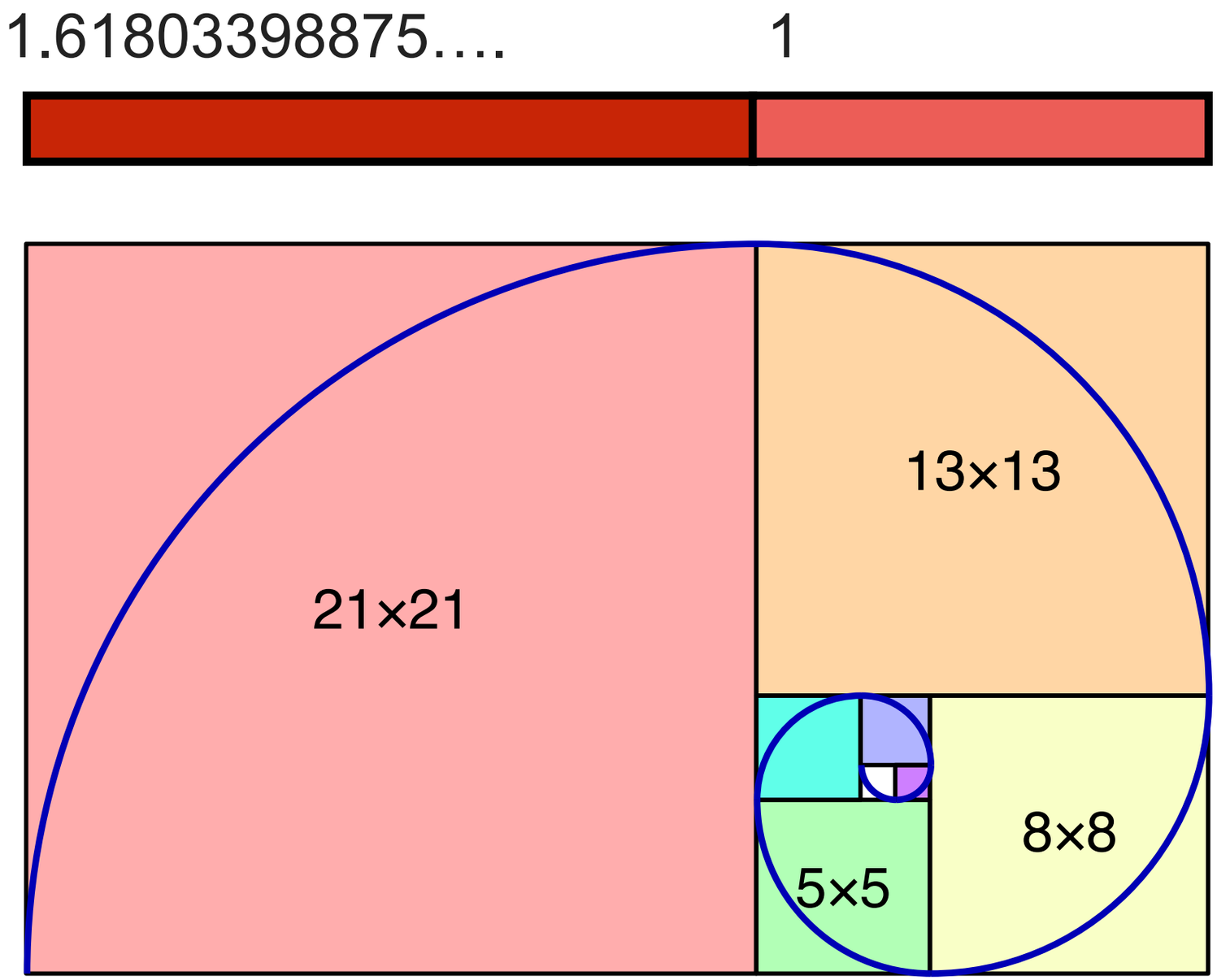
1 1 2 3 **5 8 13 21** 34 55 89



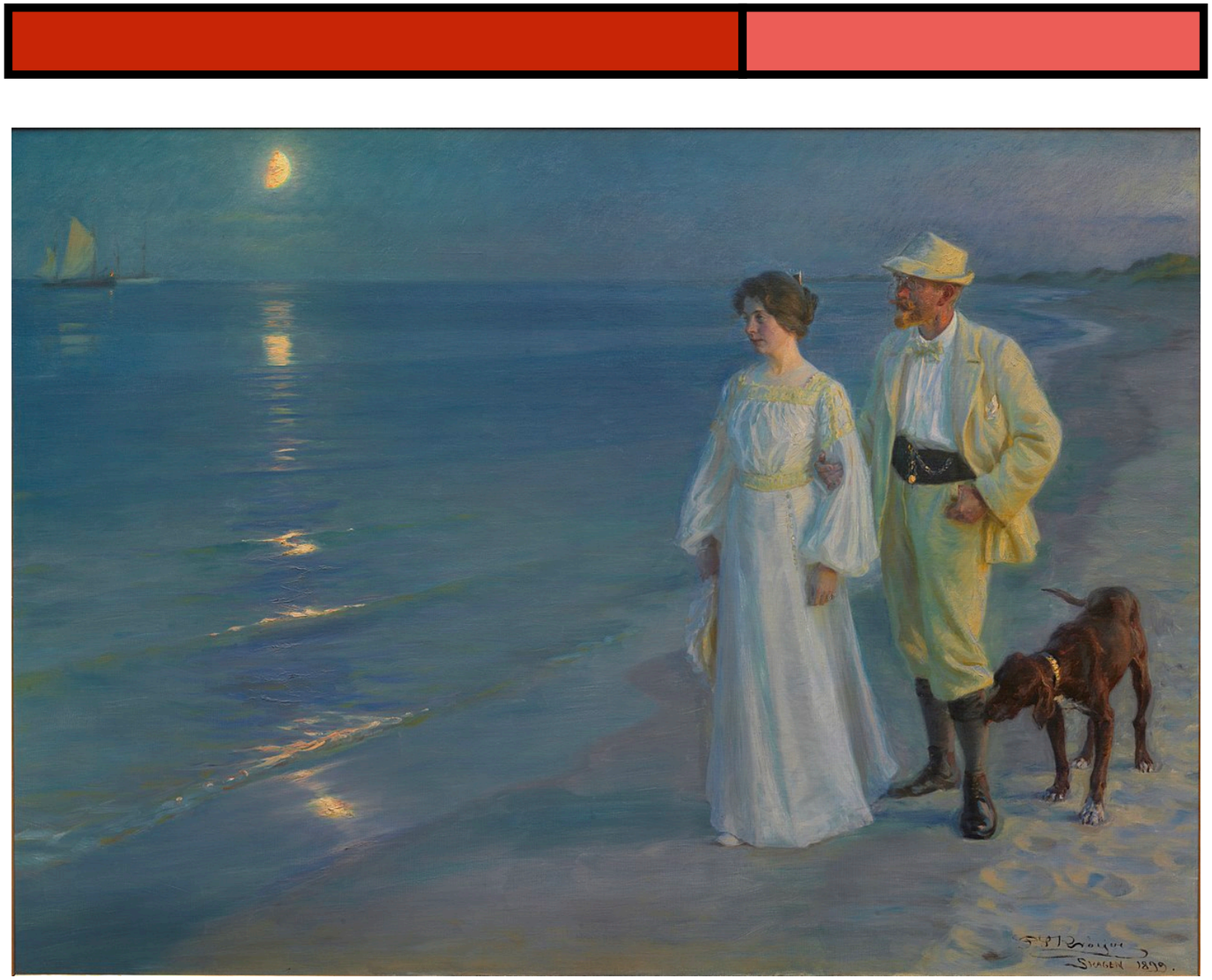
Golden ratio

$$\lim_{n\to\infty} \frac{f_n}{f_{n+1}} = \frac{1}{\phi}$$

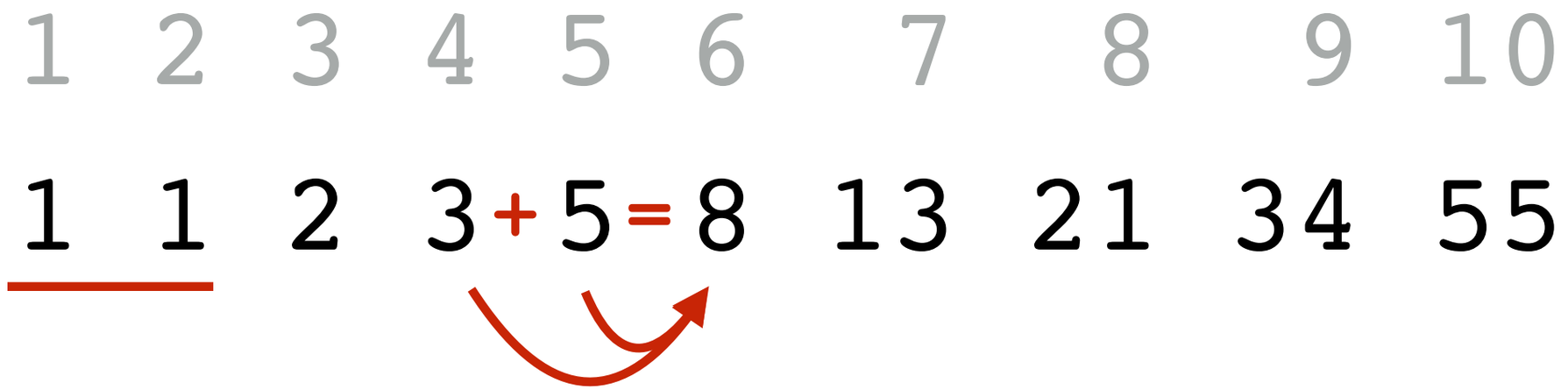




Golden ratio



Exercise



Basis:

```
fibo(1) = 1
fibo(2) = 1
```

Recursion:

```
fibo(x) = fibo(x-1) + fibo(x-2)
```

Write a function that returns a fibonacci number:

`fibo(7)` should be 13.

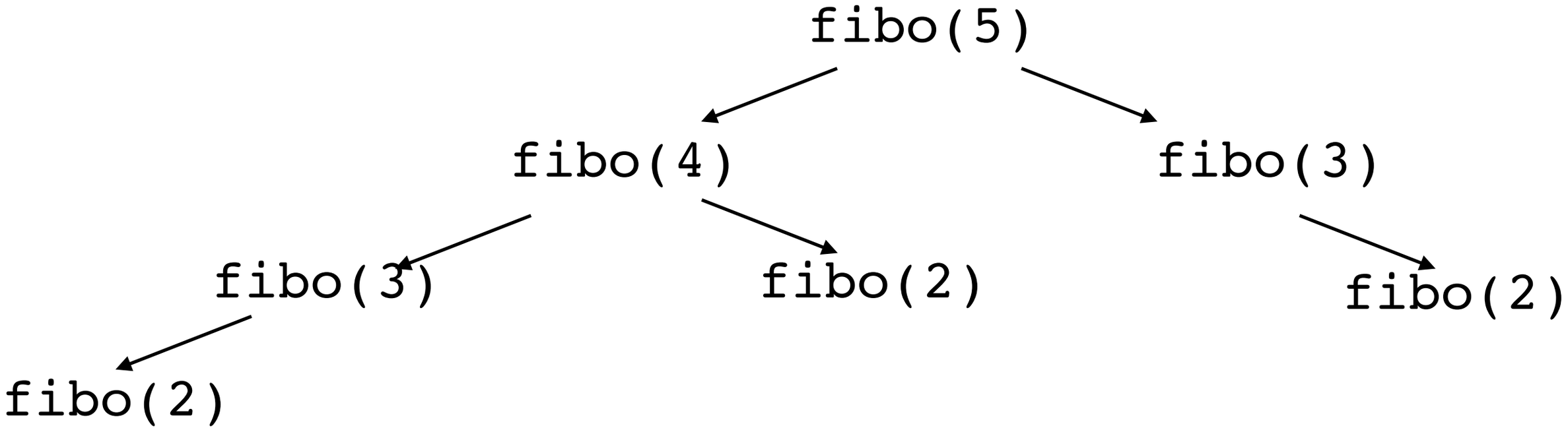
Solution

Fibonacci

Try: `fibo(30)` and `fibo(40)`

Fibonacci

Try: `fibo(30)` and `fibo(40)`



Fibonacci

```
def fibo(n):
    if n <= 2:
        return 1
    else:
        a, b = 1, 1
        for dummy in range(n-2):
            c = a + b
            a, b = b, c
        return c
```

```
      a b c
1 1 2 3+5=8 13 21 34 55
```



Try: `fibo(30)`

and `fibo(300)`